

Introduzione al C

Lezione 8

Sistemi Operativi - II Modulo (Canale A - L)

Prof. Paolo Zuliani

Dipartimento di Informatica

(Materiale didattico per cortesia del Prof. Emiliano Casalicchio)



SAPIENZA
UNIVERSITÀ DI ROMA



Agenda

- Progetti C su più file
- Il comando `make`



Costruire applicazioni in C

- Il codice sorgente di un programma C può essere suddiviso su più files
 - Riduce i tempi di compilazione (compila solo i sorgenti modificati)
 - Facilita il riutilizzo del codice (ad es. sotto forma di librerie di funzioni)
 - Facilita lo sviluppo del codice in gruppo

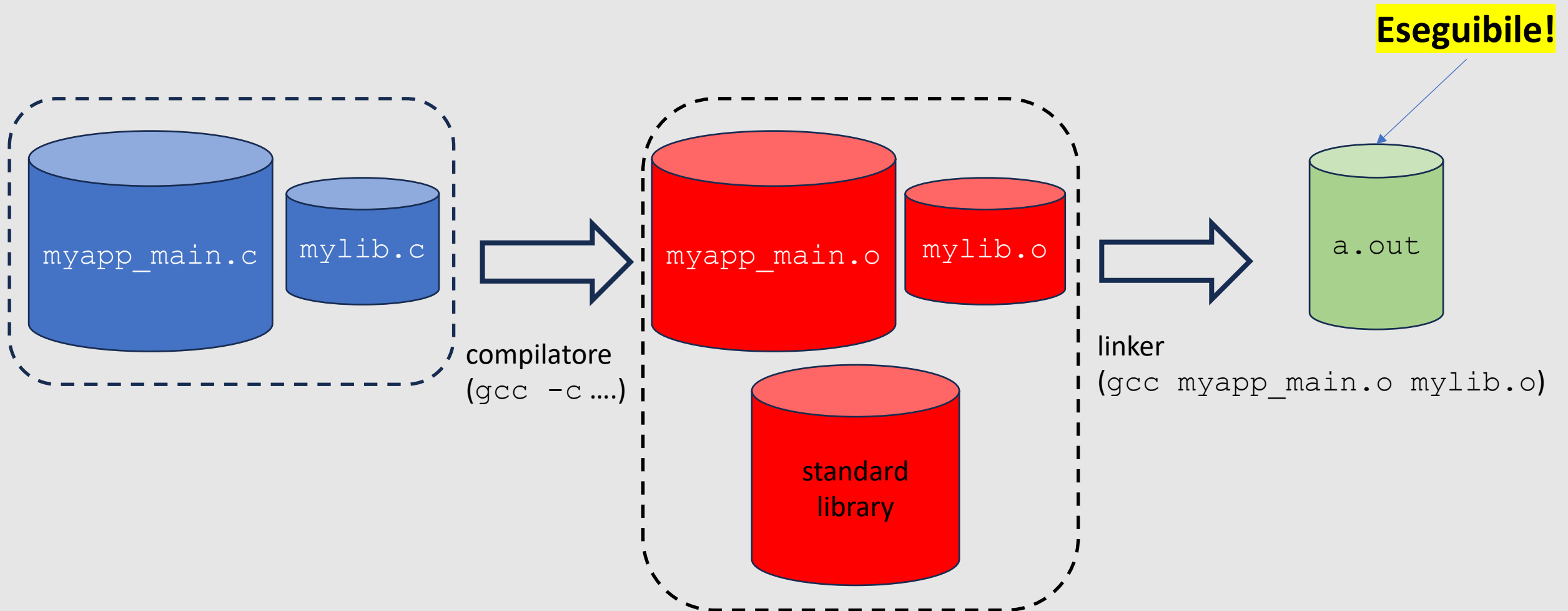
File: myapp_main.c

```
#include <stdio.h>
void main() {
    int q=0;
    q = myfunc(q);
    printf("q = %d\n", q);
}
```

File: mylib.c

```
int myfunc(int a) {
    :
    return ...
}
```

Costruire applicazioni in C



Costruire applicazioni in C

- Prima di usare una funzione/variabile in C, questa deve essere **dichiarata**. Ad esempio:
 - prima di usare `printf()` devo aggiungere `#include <stdio.h>`
 - in `stdio.h` c'è la dichiarazione di `printf` (e di molte altre funzioni)

File: myapp_main.c

```
#include <stdio.h>
void main() {
    int q=0;
    q = myfunc(q);
    printf("q = %d\n", q);
}
```

File: mylib.c

```
int myfunc(int a) {
    :
    return ...
}
```

`myfunc()` **non è dichiarata** in `myapp_main.c` e il compilatore dà errore. Come faccio ad usarla?



Costruire applicazioni in C

File: myapp_main.c

```
#include <stdio.h>

int myfunc(int a);

void main() {
    int q=0;
    q = myfunc(q);
    printf("q = %d\n", q);
}
```

File: mylib.c

```
int myfunc(int a) {
    :
    return ...
}
```

Dichiaro che in qualche file oggetto o sorgente (che verrà specificato dopo) c'è il codice di una funzione con questa intestazione.

Make e Makefile

- Meccanismo per automatizzare la compilazione di file sorgenti
- Definisce un semplice linguaggio per descrivere le relazioni tra
 - sorgenti, file oggetto ed eseguibili
- Motivazioni
 - I tempi di compilazione possono essere molto lunghi per programmi grandi (provate a compilare Google Chromium ...). Conviene ricompilare solamente se un sorgente è stato aggiornato
 - Più agevole rispetto alla compilazione manuale
- Può essere utilizzato con qualsiasi linguaggio che abbia un compilatore eseguibile da riga di comando



Make e makefile (cont.)

- In assenza di indicazioni esplicite, `make` cerca nella directory corrente un file (nell'ordine) con questi nomi:
 - `GNUmakefile`
 - `makefile`
 - `Makefile`

Make e makefile

- make attua regole della forma

Target: Prerequisites

<TAB>Recipe

<TAB>Recipe

...

<TAB>Recipe

- Queste regole sono contenute nel makefile
 - Il carattere <TAB> che precede ogni Recipe è **obbligatorio**
 - Target è tipicamente un file (ad es., un eseguibile o file oggetto)
 - Prerequisites è una lista di file o altri target
 - Recipe è un comando da eseguire (è una parte della «ricetta» per raggiungere il target)



Esempio

- La `recipe` associata ad un `target` viene eseguita quando:
 - non esiste (nella directory in cui `make` è invocato) un file di nome `target` ***e*** vi sono dei `prerequisites`
 - Il file di `target` esiste ma uno o più `prerequisites` sono stati **aggiornati dopo la sua generazione**
- Esempio di `makefile`

```
merge_sorted_lists: merge_sorted_lists.c
    gcc -Wall -Wextra -O3 merge_sorted_lists.c -o merge_sorted_lists
```

- Il `target` è l'**eseguibile** `merge_sorted_lists`
- La lista dei `prerequisites` contiene il file `merge_sorted_lists.c`
- Il `recipe` è il comando per la compilazione



Nome Makefile

- Non dobbiamo necessariamente chiamare il file `Makefile` o `makefile`
 - `make -f my_make_file`
- In un `makefile` possono esserci più `target`
- Si può specificare il `target` che si vuole eseguire
- Ad es.:
 - `make merge_sorted_lists`
- Se non si specificano `target` viene valutato solamente il primo

Phony target (azione)

- Un target senza prerequisiti è chiamato *azione* o target *phony* (finto, fasullo)

`clean:`

```
rm -f *.o merge_sorted_lists
```

- Bisogna dichiarare esplicitamente che il target è phony

```
.PHONY clean
```

- altrimenti se viene creato un file di nome `clean` nella directory in cui si esegue il `make`, l'azione sopra definita **non verrà mai eseguita**
- infatti, **un target senza prerequisiti si considera sempre aggiornato se esiste un file con quel nome**

Esecuzione di più target

ESEMPIO Makefile

```
merge_sorted_lists: merge_sorted_lists.c
    gcc -Wall -Wextra -O3 merge_sorted_lists.c \
        -o merge_sorted_lists
sort_file_int: sort_file_int.c
    gcc -Wall -Wextra -O3 sort_file_int.c \
        -o sort_file_int
.PHONY: clean
clean:
    rm -f *.o merge_sorted_lists
```

- Nel caso di più target dobbiamo esplicitare quello che vogliamo eseguire
- `make merge_sorted_list`
- `make sort_file_int`
- `make clean`



Esecuzione di più target

- Per eseguire tutti i target si crea tipicamente un target phony, chiamato **all**, il cui prerequisito è l'elenco dei target

```
all: marge_sorted_list sort_file_int  
    .PHONY: clean, all
```

Variabili

- Assegnazione base: `VAR := val`
- Accesso al contenuto di una variabile: `$ (VAR)`
- Assegnazione di una variabile
 - Espansione completa di una variabile alla lettura
`VAR := val`
 - Espansione testuale di una variabile (`val` non viene ulteriormente espansa)
`VAR = val`
 - Definizione condizionale: assegna a `VAR` il valore `val` solamente se `VAR` è vuota
`VAR = val</code`
 - Concatenazione
`VAR += val`

Esempio

```
CC := gcc
```

```
CFLAGS := -Wall -Wextra -O3
```

```
all: merge_sorted_lists \  
      sort_file_int
```

```
merge_sorted_lists: merge_sorted_lists.c
```

```
    $(CC) $(CFLAGS) merge_sorted_lists.c -o merge_sorted_lists
```

```
sort_file_int: sort_file_int.c
```

```
    $(CC) $(CFLAGS) sort_file_int.c -o sort_file_int
```



Variabili automatiche

- Variabili specifiche di make utilizzabili solo nelle ricette. Permettono di semplificarne la scrittura.
- Principali variabili
 - `^` : lista di prerequisiti del target
 - `<` : primo prerequisito del target
 - `@` : target

```
merge_sorted_lists: merge_sorted_lists.c
    $(CC) $(CFLAGS) ^ -o $@
sort_file_int: sort_file_int.c
    $(CC) $(CFLAGS) ^ -o $@
```

Esempio

```
CC := gcc
```

```
CFLAGS := -Wall -Wextra -O3
```

```
→ PROGS := merge_sorted_lists \  
            sort_file_int
```

```
all: $(PROGS)
```

```
→ merge_sorted_lists: merge_sorted_lists.c  
    $(CC) $(CFLAGS) $^ -o $@
```

```
→ sort_file_int: sort_file_int.c  
    $(CC) $(CFLAGS) $^ -o $@
```

```
.PHONY: clean
```

```
clean:
```

```
    rm -f *.o $(PROGS)
```



Esempio

Makefile

```
CFLAGS := -O2
```

```
CC := gcc
```

```
helloworld : helloworld.o myfunction.o
```

```
$(CC) $^ -o $@
```

```
%.o : %.c
```

```
$(CC) $(CFLAGS) -c $< -o $@
```

```
.PHONY: clean
```

```
clean:
```

```
rm -f *.o
```

```
$make helloworld
```

```
gcc -O2 -c helloworld.c -o helloworld.o
```

```
gcc -O2 -c myfunction.c -o myfunction.o
```

```
gcc helloworld.o myfunction.o -o helloworld
```

Esercizio

- Usando `getch.c`, scrivere un programma C che legga da tastiera una stringa arbitraria e stampi a video se la stringa è palindroma
- Strutturare il codice in almeno **tre** file sorgenti C:
 - un file per la funzione `main()`
 - uno per una funzione che legge la stringa
 - uno per una funzione che controlli se una stringa è palindroma
- **PRIMA DI SCRIVERE IL CODICE C** scrivere un `makefile` per generare l'eseguibile
 - L'eseguibile deve essere generato eseguendo `make` senza alcun parametro
 - Aggiungere un target *clean* per cancellare tutti i file oggetto `.o`
- **Usare `make` invece di invocare `gcc etc.` durante lo sviluppo!!**

Regole e Variabili implicite

- make possiede un insieme di regole implicite che coprono casi comuni (ad es., la generazione di un eseguibile a partire da un sorgente C)
- Permettono di scrivere regole formate dal solo target o da target e prerequisiti, senza una recipe esplicita
- Esempio:
 - `main: main.c list.h list.c`
 - crea un eseguibile `main` a partire da prerequisiti
- Se il target dipende da un solo prerequisito con il suo stesso nome (a meno dell'estensione), il prerequisito si può *omettere*
 - make cerca un file con il nome del target e sceglie il recipe implicito da utilizzare in base alla sua estensione
 - `es:merge_sorted_lists:`

Regole e Variabili implicite (cont.)

- Le regole implicite usano variabili implicite
- Essendo ridefinibili dall'utente permettono di "controllare" l'esecuzione delle recipe implicite
- Principali variabili (modificabili dall'utente)
 - CC : compilatore
 - CFLAGS : flag per la fase di compilazione
 - LDFLAGS : flag per il linker (-L)
 - LDLIBS : flag per il linker: aggiunge librerie (-l<libname>)
- La recipe **implicita** eseguita per la compilazione di un eseguibile `x` a partire da un singolo sorgente `x.c` è:
 - `$(CC) $(CFLAGS) $(LDFLAGS) x.c $(LDLIBS) -o x`
- Quindi ad esempio il target `main`: `main.c list.h list.c` *eseguirà la regola implicita*
 - `$(CC) $(CFLAGS) $(LDFLAGS) main.c list.h list.c $(LDLIBS) -o main`

Esempio

```
CC := gcc
CFLAGS := -Wall -Wextra -O3

PROGS := merge_sorted_lists \
         sort_file_int \
         main

all: $(PROGS)

main: main.c list.h list.c

.PHONY: all clean
clean:
    rm -f *.o $(PROGS)
```

